

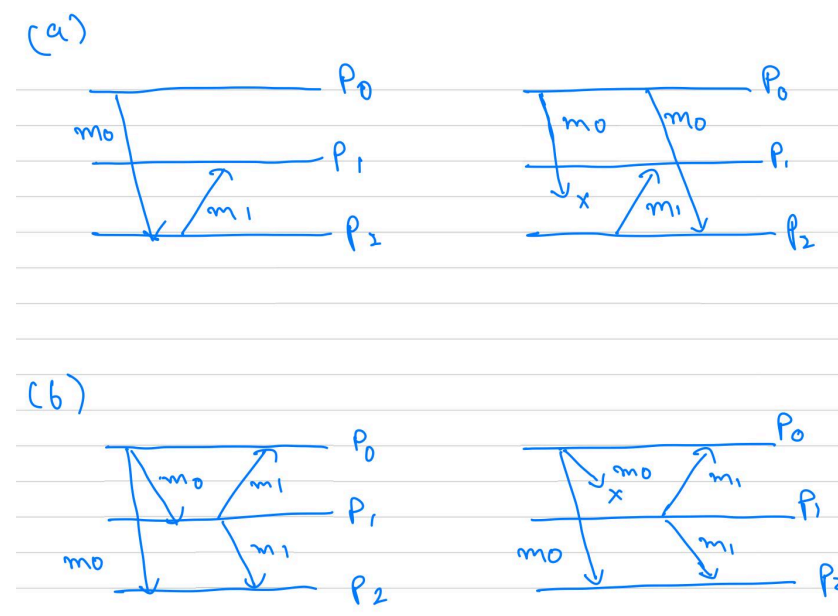
Question 1:

- The two pairs of messages could be: <bl> and <nq>.
- The FIFO violation is between messages: <aj> and <ch> as event j should have happened before h.
- There is a causal violation between <ch> and <mk> as event k should have happened before h.
- The two possible ordering for messages that preserve causal ordering are:
 - <aj>, <bl>, <ch>, <pd>, <mk>, <nq>, <ie>, <os>, <rf>
 - <aj>, <bl>, <ch>, <mk>, <pd>, <nq>, <ie>, <os>, <rf>

Question 2:

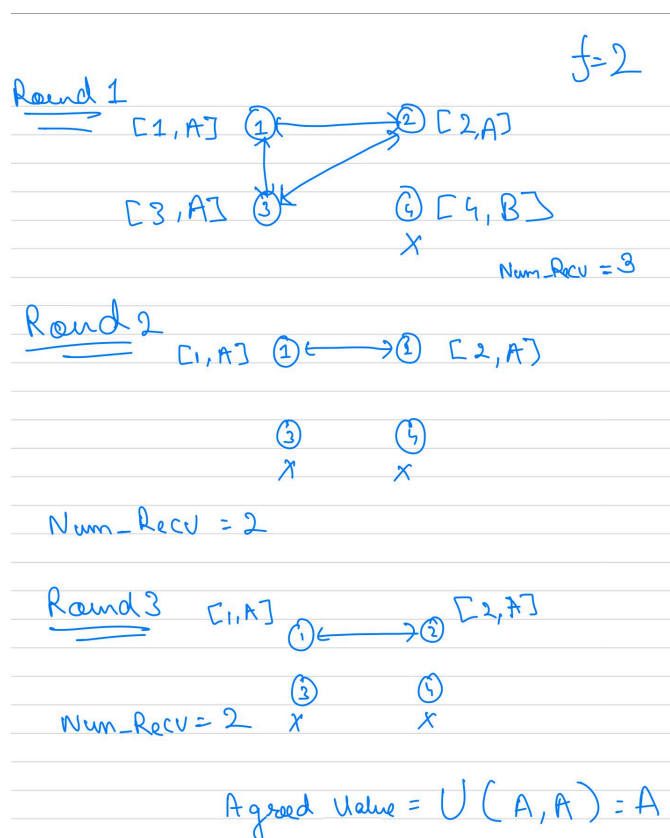
(a) In case of unicast messages for $n > 2$ processes, reliable FIFO messaging does not ensure causal ordering. In the example figure below in part (a), we can see a case where the message m_0 from P_0 to P_2 can fail and due to retry attempts, the message m_0 could reach P_2 later than when message m_1 got delivered to P_1 . The figure on the left shows the ideal condition, where m_0 should arrive at P_2 before m_1 gets delivered to P_1 . The figure on the right shows causal ordering failure condition.

(b) In case of multicast messages as well, we cannot preserve causal ordering. The figure (b) shows two scenarios involving multi-cast messages similar to (a). The left side of the figure shows the ideal condition where message m_1 should be delivered to process P_2 after m_0 . The right side of the figure shows the scenario where message m_0 from process P_0 got lost and process P_1 sent the message m_1 to process P_2 without receiving the retried message m_0 .



Question 3:

In the flooding algorithm, the consensus cannot be reached in f or fewer rounds if we consider the worst case scenario. In the worst case, we assume that for every round, one node will fail during the round. This means that for f rounds, we will not be able to reach any consensus as we get less number of receives for each non-failed node than the previous round. We can only come to consensus when we receive from the same set of nodes for 2 consecutive rounds. This can only happen at round $f + 1$. Below is the example depicting 4 nodes, with states $\langle 1, A \rangle$, $\langle 2, A \rangle$, $\langle 3, A \rangle$, and $\langle 4, B \rangle$.



In the first round, node 4 dies and so the number of received messages is less than the active population before the round. Similarly in the second round, Node 3 dies. Lastly in the third round, no node dies and the number of received messages is equal to the number of active nodes before the round. Hence, consensus can be reached.

Question 4:

Safety properties suggest that nothing bad will ever happen to the application components.

Liveness properties suggest that something good will eventually happen.

For A: A distributed repository of personnel data that provides personnel information to the authorized users.

Safety Property: Personnel data will never be provided to any unauthorized users.

Liveness Property: Any authorized user requesting the data will eventually get it.

For B: A highly available airline reservation system that allows clients to book seats.

Safety Property:

- (i) The airline reservation system will never crash while the client is booking the seats.
- (ii) Concurrent clients will never cause the system to go into a deadlock state.

Liveness Property:

- (i) The client's request to book seats will eventually process successfully.
- (ii) Concurrent client requests will eventually be able to acquire the seats.

For C: An air traffic control system that manages plane landings and takeoffs.

Safety Property:

- (i) No two planes are scheduled for landing at the same time.
- (ii) No two planes are scheduled for take-off at the same time.
- (iii) No two planes are scheduled for concurrent landing and take-off at the same time.

Liveness Property:

- (i) Concurrent requests will eventually be scheduled sequentially with some interval in between to ensure safety.

Question 5:

The RAFT paper describes the election-restriction criteria which only lets a particular candidate to be elected as leader only if it contains all the previously committed messages. The log entries only flow in one direction. In this problem, as S3, S4, and S5 commit entries in term 11, S1 would not have this log in its state and hence would not be able to become the leader.

In the provided example, either S4 or S5 can become the next leader as they have the latest committed entries. Even though S1 and S2 conducted multiple election rounds, they miss the committed message from term 11. S1 and S2 would be forced to become followers so that the newly elected leader can replicate the missing entries to S1 and S2.